



Inspiring Excellence

CSE321 : Operating System

Farhan Sadik

24101406, Sec-09

Introduction

Multithreading, makes it possible to execute more than one task within a single program or process at the same time. All the programming languages provide their own library or API to create and manage threads, however behind the scenes they are all utilizing the operating system in some way or another. For this report I have implemented comparison functions to test three different threading systems: Python threading module, pthreads (C/C++), and Swift threading. For each, I have explained the library implementation, how it relates to the operating system and how the userlevel threads are mapped to kernellevel threads. Eventually, I've done a bit of comparison of their efficiency and overhead under various scenarios.

Python Threads

Library Implementation

Python has two modules for threading: 1) threading and 2) queue. First, the low-level module is thread, and, as an overlay on that, there is the high-level threading module, which is the one that the programmers typically use. The threading module provides a thread class that can be used to create and start new threads; also synchronization primitives such as Lock, RLock, Semaphore, Event, Condition, and Barrier (Python Software Foundation, n.d.). Creating a thread is very easy, its just the passing of something like a function to the Thread object and calling start().

Python threads are Preemptive and not Cooperative threads. A thread does not have to give up control, the operating system may pause it at anytime. But one special rule, with the standard CPython interpreter, is called the Global Interpreter Lock (GIL). Even if the computer has a large number of CPU cores, only one thread per GIL can be executing Python bytecode instructions. Clearly, the interpreter induces the GILs to drop after a short

period of time (default is 0.005 seconds) for other threads to run (Python Software Foundation, n.d.). So switching is still preemptive but the ability to execute python itself in parallel is prevented by the GIL. It should be noted that the last versions of Python (3.13 and later) helped to introduce an optional “free-threaded” build which disables the GIL, but the normal builds still preserve the GIL.

OS Integration

Python does not have an implementation of its own scheduler. All Python threads are true OS threads. CPython uses the POSIX pthread API to create the threads on Linux and macOS, and the native Windows thread API on Windows. This is why the threading module is cross-platform, the same Python code works in Windows, Linux and macOS and Python abstracts the differences specifically to do with threads.

The Python threads are actual OS threads and receive all the typical OS performance optimizations for them, including fast synchronization primitives (for example futex on Linux), scheduling to multiple cores, thread priorities, etc. But the benefit of multi core is mostly eliminated by the GIL for CPU bound python code. That's why python threads are primarily suitable for I/O-related tasks (network response, file read, awaiting, etc.) since the GIL is released in blocking I/O calls, thus allowing other threads to run (Python Software Foundation, n.d.).

Mapping between user level and kernel level threads

You can see that in Python, there is a single character for every single character. All the Python threads are backed by their own actual process-level threads, which are managed by the OS kernel (Silberschatz et al., 2018). Python has no scheduling of its own, only more control is the GIL, which determines which thread may execute Python bytecode at a given moment. So it's a kernel-level thread with an interpreter level lock on top of it. The overhead of creating a Python thread is that of creating an native OS thread and some book keeping related to the interpreter. This is also the case for CPU-bound parallel programs, which Python programmers typically implement using the multiprocessing module, since each process does not acquire the GIL.

POSIX Pthreads

Library Implementation

Pthreads (POSIX threads) is not the product of any single company, rather it is an API formally specified by IEEE in the POSIX standard (IEEE 1003.1c). It's a C library, `pthread.h` is the header for the library. The standard specifies approximately 100 functions; the most important functions for managing threads are `pthread_create()`, `pthread_join()`, `pthread_exit()`, and `pthread_detach()`. It provides mutexes (`pthread_mutex_t`), condition variables (`pthread_cond_t`), read-write locks and barriers (Butenhof, 1997). Pthreads provides very fine-grained control, such as stack size, scheduling policy and thread priority, using attributes.

Preemptive (all common systems - Pthreads) The kernel scheduler might stop any thread when its time slice expires and allow another thread to use the CPU. Note that a pthread need not be willing to give up the CPU, although there is a function `sched_yield()` that can be called by a willing thread.

OS Integration

Of the three, the one that's most like the operating system is Pthreads, which is just a thin layer over operating-system provided system calls. On Linux, it is called Native POSIX Thread Library or NPTL, an integral part of glibc, and generates threads using `clone()` system call. Since pthreads is implemented on top of the native threads of BSD systems and Mac OS, it is effectively the same thing on those operating systems. Pthreads is standardized and will work on all POSIX systems (such as Linux, Mac, BSD, Solaris, etc); however pthreads are not supported natively on Windows and require a compatibility layer (pthreads-win32) or must be used directly on Windows (Barney, 2023). Thus it's portable, and only in the Unix family.

The OS provides lots of optimizations for pthreads. Mutexes on Linux are based on fast user-space locks (called futexes), thus in the non-contention case, locking never invades the kernel space. NPTL was created such that creation and destruction of many threads is fast,

the kernel scheduler is O(1)/CFS based, thus can handle thousands of threads. The kernel also attempts to confine a thread to a particular CPU core (processor affinity) in order to take better advantage of the cache.

Mapping between user level and kernel level threads

In the one-to-one model, which is most common today, each pthread is one kernel thread, and the kernel scheduler deals directly with all of the threads. Some systems (e.g. old Solaris) used to have a many-to-many model for pthreads, but that has been scrapped in favor of the simpler 1:1 model: modern kernels are fast enough! Real parallelism, 8 pthreads can actually execute parallel with an 8 cores machine. The drawback is that each thread creation, a context switch and each blocking operation will have to be handled by the kernel, which is heavier than pure user-level threads. But for CPU bound work pthreads generally yields the highest performance of the three systems reported here, as there is no interpreter and no lock (such as the GIL).

Swift Threads

Library Implementation

Swift has actually three levels of threading tools. There is the Thread class of the Foundation framework, which is used to handle each thread directly, like pthreads but with some convenience (Apple Inc., n.d.-b). The second, and the more widely used one, is Grand Central Dispatch (GCD, also known as libdispatch). Programmers doesn't create threads manually. Instead, small tasks (closures) are pushed into dispatch queues and the providence of a pool of worker threads is handled automatically (Apple Inc., n.d.-a). Third, modern Swift (5.5 and later) has structured concurrency with async/await, actors, and tasks, which runs on a cooperative thread pool.

The underlying OS threads are always preemptive the kernel can stop any of them. Thus, thread and GCD tasks are preemptive scheduled. However, it is interesting how the current Swift Concurrency model is cooperative: it is cooperative at the language level, the runtime operates with around one thread per CPU core, and an async task yields its thread only at

await isolation points. So Swift combines both styles, cooperative task switching atop of preemptive kernel threads.

OS Integration

The Swift threading system runs atop of Mach kernel threads and the pthread layer of Darwin on Apple platforms (macOS, iOS). GCD is also an integrated part of the OS, the kernel itself can determine the number of worker threads to create based on system load, and includes Quality of Service (QoS) classes, allowing background activities to run with low priority and conserve battery (Apple Inc., n.d.-a). The great thing about this tight integration is that the thread pool doesn't start generating too many threads ("thread explosion") and the scheduler becomes aware of the priority of each queue.

The issue is that Swift itself is cross-platform now (same with libdispatch, which has been ported to Linux and Windows), although the best support and optimization is still Apple platforms, but in practice Swift threading seems more platform dependent than Python or pthreads. The implementation of Foundation's Thread on Linux uses pthreads.

Mapping between User Level and Kernel Level Threads

It can depend on the layer used for mapping. The Foundation Thread class is a pure one-to-one mapping where one object = one kernel thread. GCD is closer to being many-to-many (aka hybrid): many provided tasks are multiplexed on a miniscule number of kernel-worker threads, and the number of threads in the pool is manipulated by the OS. Swift Concurrency is also a hybrid/cooperative model: a lot of small tasks are scheduled on a static set of kernel threads (one per core) and that switching between tasks is done in user space when they awaits, and not in the kernel when doing a switch that is much more expensive (Apple Inc., n.d.-a; Silberschatz et al., 2018). This means that Swift does have a very small overhead for a large number of concurrent tasks a useful thing for serving and UI applications.

Comparison and Discussion of Efficiency and Overhead

Factor	Python threading	POSIX pthreads	Swift (Thread / GCD / Concurrency)
Language / level	High-level (interpreted)	Low-level C API	High-level, multiple layers
Multitasking style	Preemptive (limited by GIL)	Preemptive	Preemptive (Thread, GCD); cooperative tasks in Swift Concurrency
Thread mapping	1:1 (kernel threads + GIL)	1:1	1:1 for Thread; hybrid pool (M:N style) for GCD and Concurrency
Platform	Cross-platform	POSIX systems (not native on Windows)	Best on Apple platforms; Linux/Windows supported
True parallelism of code	No (with GIL); yes for I/O and native extensions	Yes	Yes
Creation/switch overhead	OS thread cost + interpreter cost	OS thread cost (lowest of the three for raw control)	Very low per task (pool reuses threads)
Best use case	I/O-bound programs, simple scripts	CPU-bound, system programming, servers	App development, many small concurrent tasks

These three libraries excel in different things (from an efficiency standpoint). The winner is pthreads for CPU-bound work (heavy calculation), using none of the interpreter overhead, with this type of work running parallel on all cores with native code. Python is the worst here due to the GIL: regardless of 8 threads, only one of them runs python code so when you're working in the CPU bound mode, Python threading could be even less efficient with one thread than using it without threading at all due to switching the locks. Swift works on CPU like pthreads as it compiles to native code.

All three can be used for I/O-bound Work (a lot of network / disk access). Python threads do not cause any problems here as the GIL is released while waiting. However, if there are many many (thousands) operations that execute concurrently, models that create an individual kernel thread per operation (raw pthreads, Python threads, Foundation Thread) are expensive because each kernel thread requires its own stack (which often consists of megabytes of memory) and require a kernel context switch for each switch into and out of operation. In such a situation, GCD and Swift Concurrency are significantly more efficient since thousands of light tasks are multiplexed onto a few kernel threads and this multiplexing happens in user space (Apple Inc., n.d.-a).

On Manual programming and Safety level, pthreads provides the greatest freedom but also most opportunities for error (Manual Joins, Manual handling of Mutex, Data Races, etc.) Python is easy to write but least powerful. I believe that Swift is in the sweet spot, that GCD and async/await do the thread management and the actor model helps to avoid data races at compile time.

Conclusion

To sum up, eventually all three threading systems rely on kernel level threads, but present a different interface. Pthreads is a thin standard API that provides maximum control and true parallelism on POSIX systems in a preemptive 1:1 manner. The same type of kernel threads are wrapped into an very easy cross-platform API by Python's threading module, which can only be used for concurrency (mostly I/O) and not for real parallelism thanks to the GIL. Swift provides three types of programming models: a 1:1 Thread class; the built-in OS GCD thread pool; and a

new, modern cooperative task system with very low overhead, that combines user-kernel scheduling. Hence the "best" library is whatever is required for each particular case – pthreads where raw CPU and control is driving the performance, simple I/O-bound scripts in Python, for apps with many concurrent tasks, Swift's GCD/Concurrency.

References

Apple Inc. (n.d.-a). *Dispatch*. Apple Developer Documentation.

<https://developer.apple.com/documentation/dispatch>

Apple Inc. (n.d.-b). *Thread*. Apple Developer Documentation.

<https://developer.apple.com/documentation/foundation/thread>

Barney, B. (2023). *POSIX threads programming*. Lawrence Livermore National

Laboratory. <https://hpc-tutorials.llnl.gov/posix/>

Butenhof, D. R. (1997). *Programming with POSIX threads*. Addison-Wesley.

Python Software Foundation. (n.d.). *threading — Thread-based parallelism*. Python 3 documentation. <https://docs.python.org/3/library/threading.html>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.